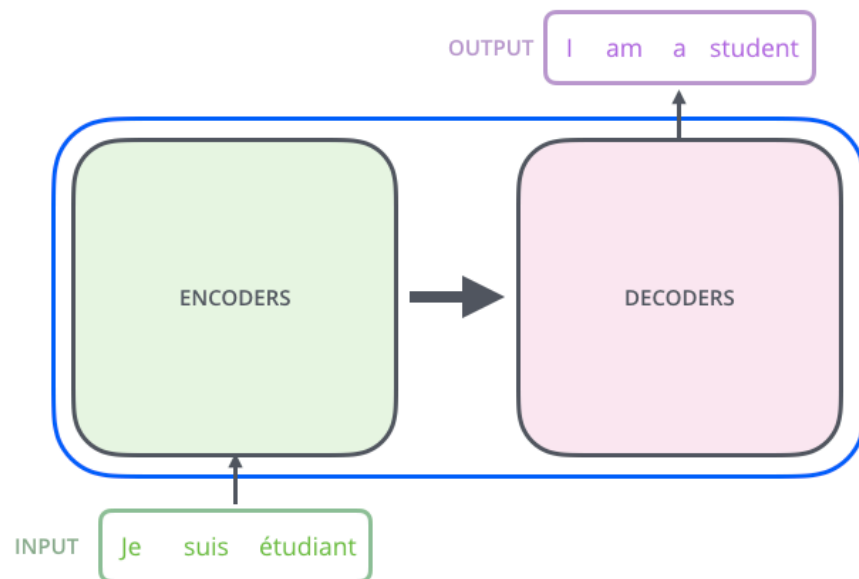


Transformers

Quick Recap

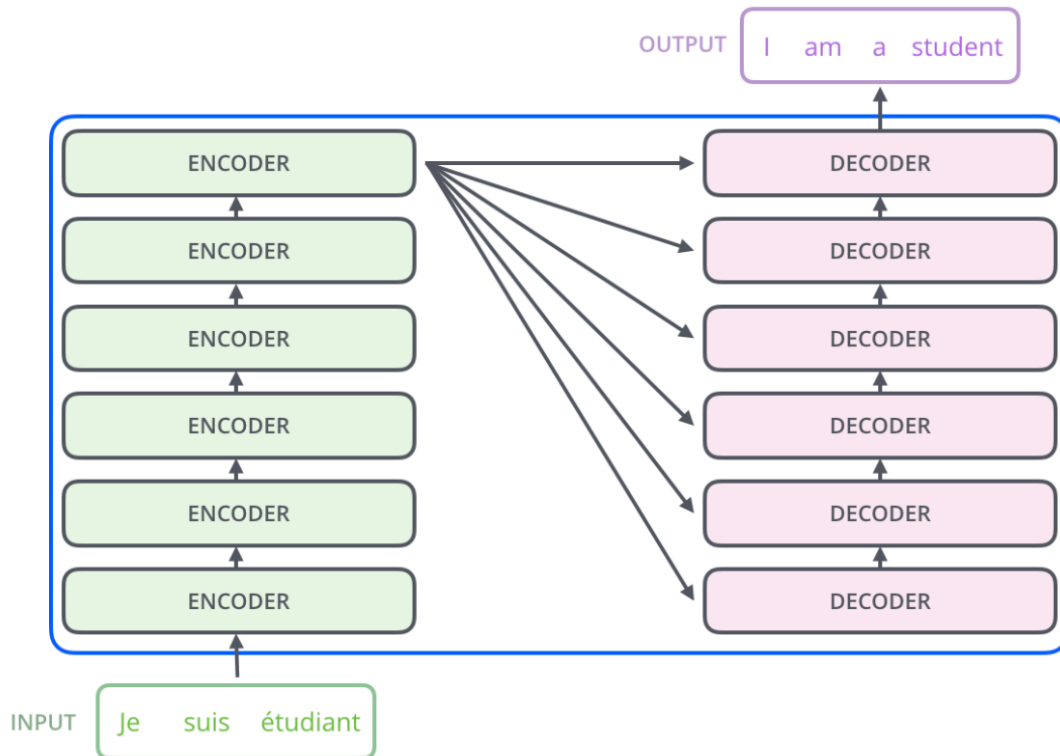
Transformers are formed of Encoder and Decoder



For language conversion , Encoders are fed one language and decoder outputs another language.

Encoders are used to understand the context while decoder are used to generate context.

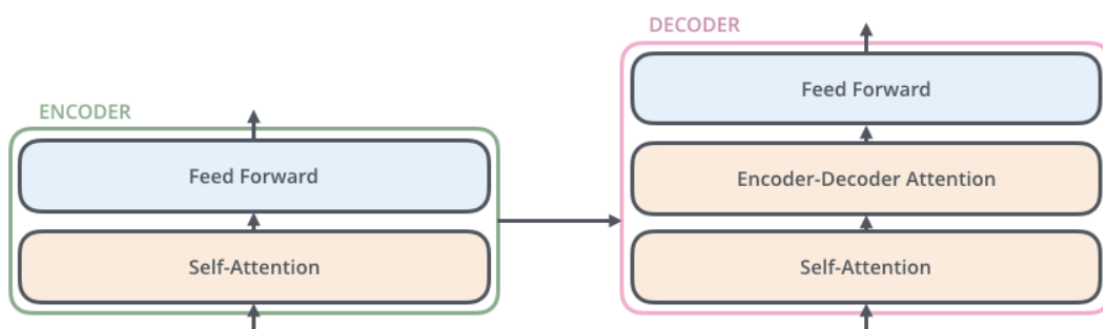
Encoders and decoders contains a number of itself within them and there is no hard and fast rule to how many we can implement.



Each encoder is broken down into two sub layers

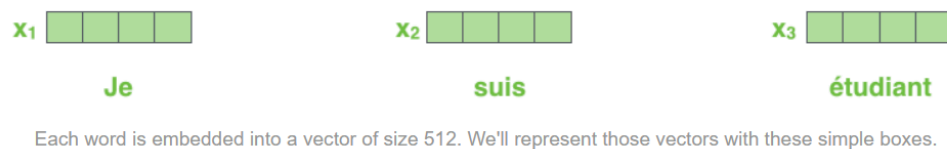
- Self Attention
- Feed Forward Network

Decoder also has both of these layers but there is an extra layer called "Encoder Decoder Attention" which helps decoder focus on relevant part of the input sentence.



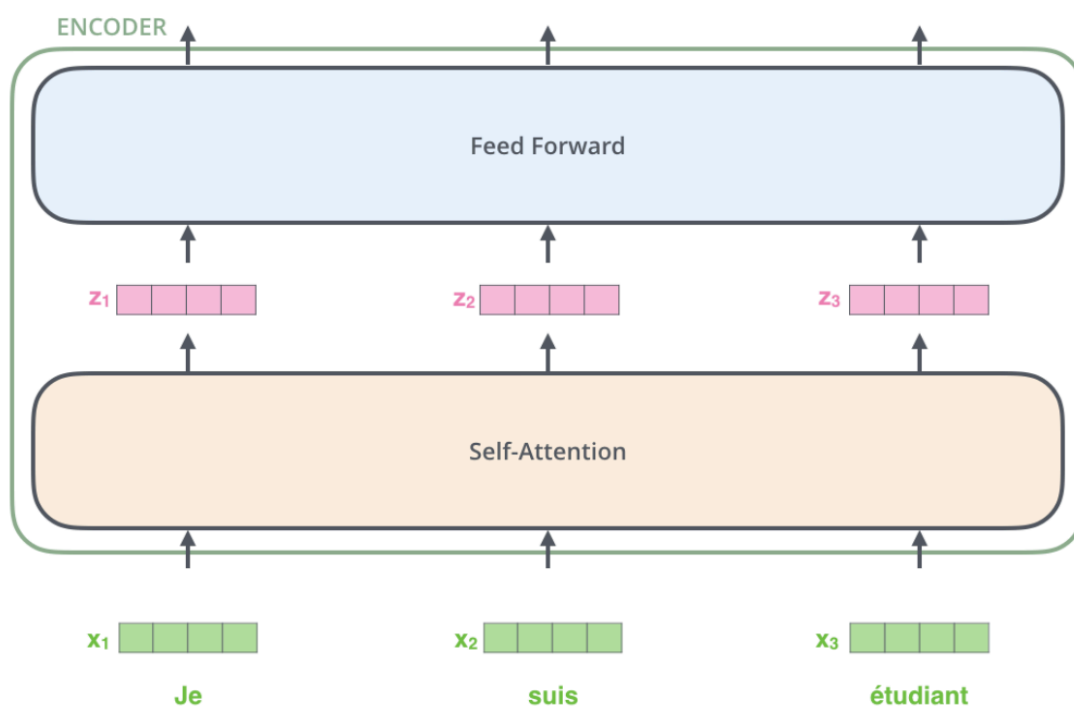
Introducing Tensors

The words are embedded into vectors which are then passed to the encoders.



The embedding only happens in the bottom layer, and vector size is taken to be 512

After embedding the word, each word flows through two layers of encoder.



We see that word in each position flows through its own path in the encoder. There are dependencies between these paths and self-attention layer.

Self Attention

In simple words , while reading a long sentence let's say , "A dog goes on a walk as it is feeling board sitting at home."

Here what does it refers to ,it refers to the "dog which is on a walk who felt board sitting at home".

When the model is processing the word "it", self-attention allows it to associate "it" with "animal".

Implementing Self-Attention

Step 01 :

- Create 3 vector from each encoder input vector having smaller dimension 64
 - Query Vector
 - Key Vector
 - Value Vector

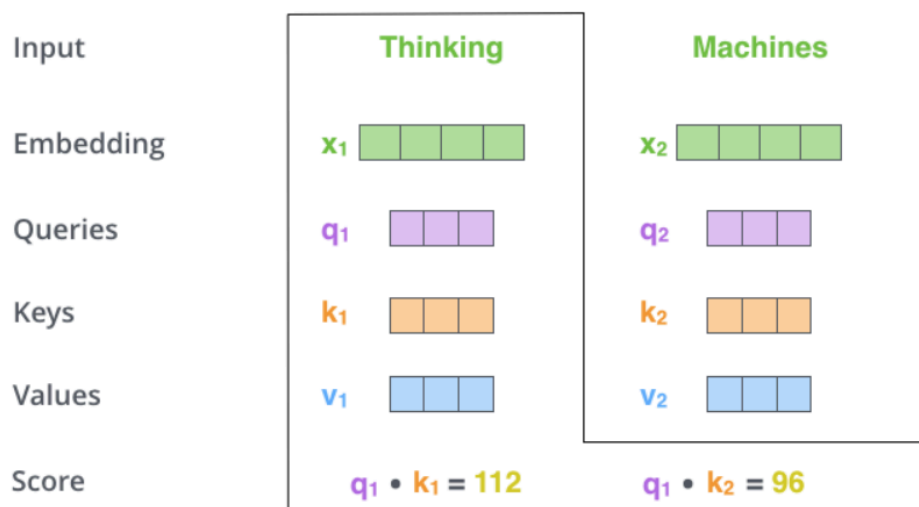
Step 02 :

Calculate scores

Let's say we are calculating attention score for the word "Thinking" .We need to score each word of the input sentence against this word . This score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.

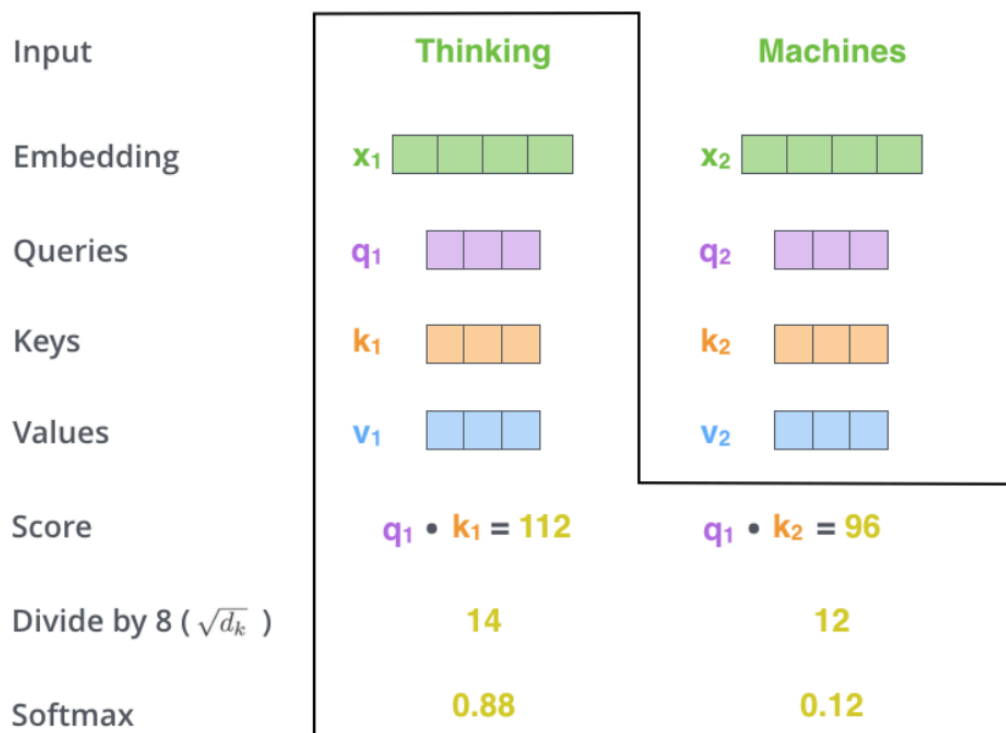
The score is calculated by taking dot product of ""query vector" with the key vector of the respective word we are scoring, so if we are processing the self attention for the word of position #1 .

The first score would be the dot product of q_1 and k_1 . The second would be the dpt product of q_1 and k_2 .



Step 03 and 04 :

Divide the score by 8 (square root of dimension of key vector used) and pass the result to soft-max , soft max normalizes the scores and makes sure they add up to 1 .



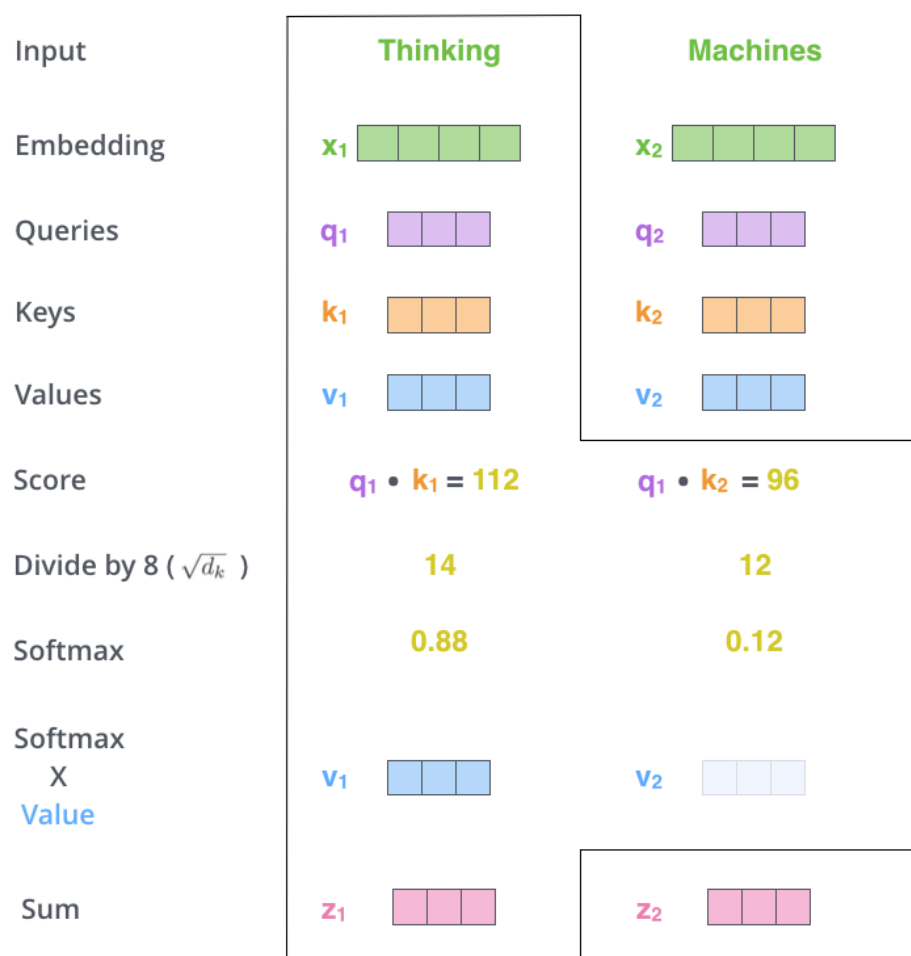
The soft-max score determines how much of each word is expressed at this position. Clearly the current word will have the correct score.

Step 05 :

Multiply each value vector by soft-max score (in order to sum them up). The main objective here is to keep intact the values of the word(s) we want to focus on and ,drown out irrelevant words we want to focus on. [say by multiplying them with example : 0.0001].

Step 06 :

Sum up the weighted value vectors . This produces the output of the self attention layer at this position (for the first word).

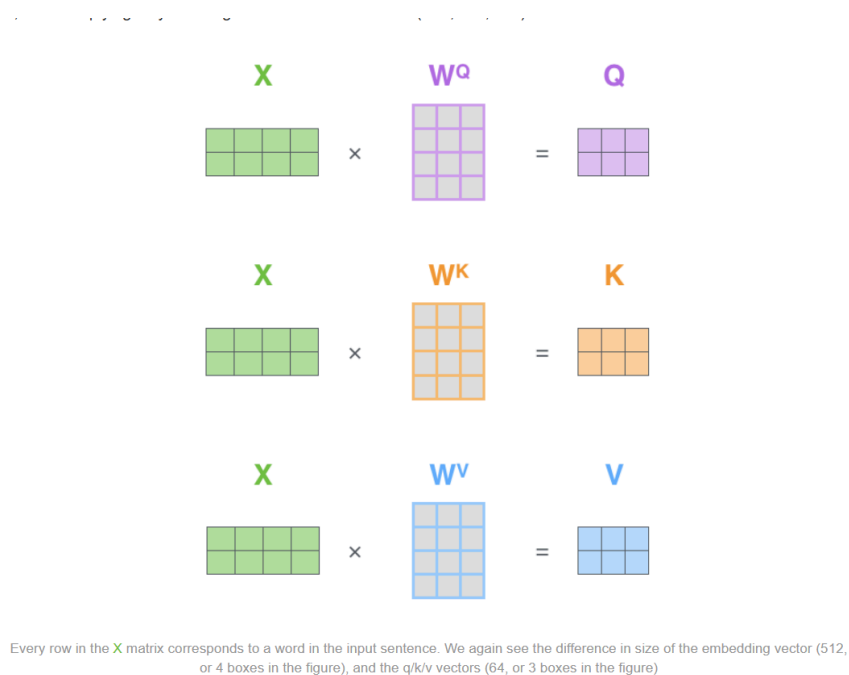


This shows that among two words , less relevant word is discarded while keeping the more relevant words

Matric Calculation of Self-Attention

1. Calculate the Query Value of Matrices.

We pack our embedding of vector (512) into matrix X and multiply by weights (W^Q, W^K, W^V). Weights are either pre trained or chosen in random at the time of model initialization and optimized through iteration.



2. We condense the steps 2 to 6 in one formula to calculate the outputs of the self attention layer.

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{matrix} \square & \square \\ \square & \square \\ \square & \square \end{matrix} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix}$$

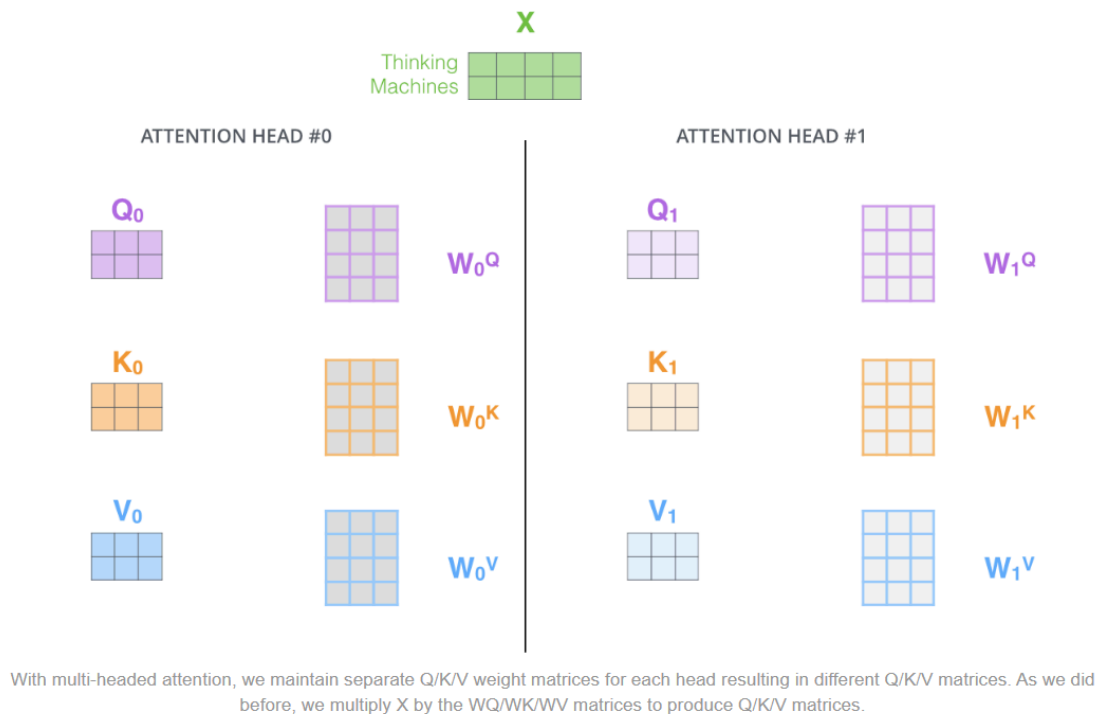
$$= \begin{matrix} \text{Z} \\ \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix}$$

The self-attention calculation in matrix form

Multi-Head Attention

Multi-Head Attention was introduced improving the performance of attention layer in two ways:

1. Multi-head attention lets each head focus on different parts of the sentence (e.g., helping "it" correctly refer to "animal" instead of "street"). in the example "The animal didn't cross the street because it was too tired".
2. It creates multiple independent representation subspaces by using 8 separate sets of Q/K/V weight matrices, each learning a different way to view the input.



3. We calculate attention separately so we end up with 8 Z's , but the feed forward network is expecting one Z , so we concatenate the values of Z and multiply it with another weighted matrix W_o .

1) Concatenate all the attention heads

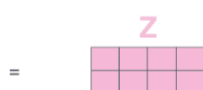


2) Multiply with a weight matrix W^o that was trained jointly with the model

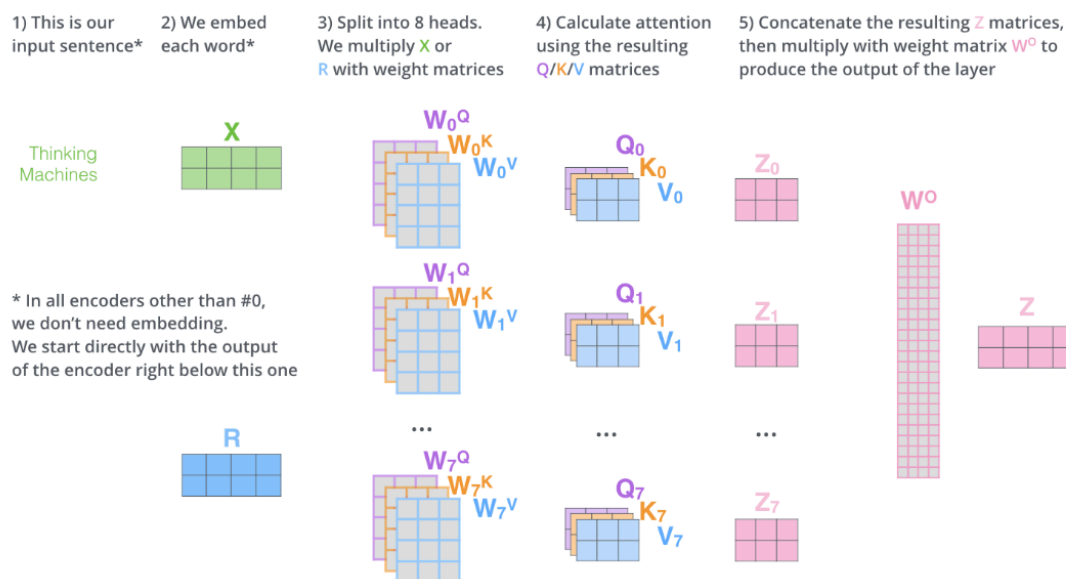
X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

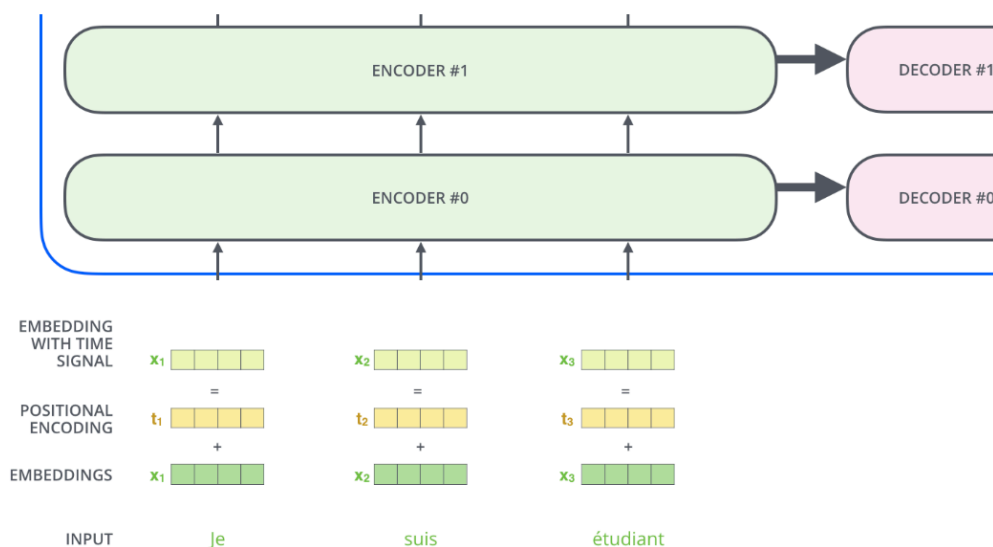


In summery , the entire system may work like the following order.



Representing the order of Sequence Using Positional Encoding.

To account for order of words in the input sequence ,transformer adds a vector to each input embedding which follow a specific pattern that model learns which help it determine the position of each word or the distance between words in the sequence.



To give the model a sense of the order of the words, we add positional encoding vectors – the values of which follow a specific pattern.

If we assume the embedding has dimensionality 4, the actual positional encoding would look like this:



A real example of positional encoding with a toy embedding size of 4

The positional encodings are calculated by sine or cosine functions and added to the vector embedding in order for the model to learn “WHAT” and “WHERE”.

- These are **not learned** — they are **fixed mathematical values** calculated using **sine and cosine functions**.
- For each position (1st word, 2nd word, 3rd word), the sine/cosine formula generates a unique vector of the same size (4 in this toy example).
- The formula used is the one from the original paper (Vaswani et al., 2017):

text

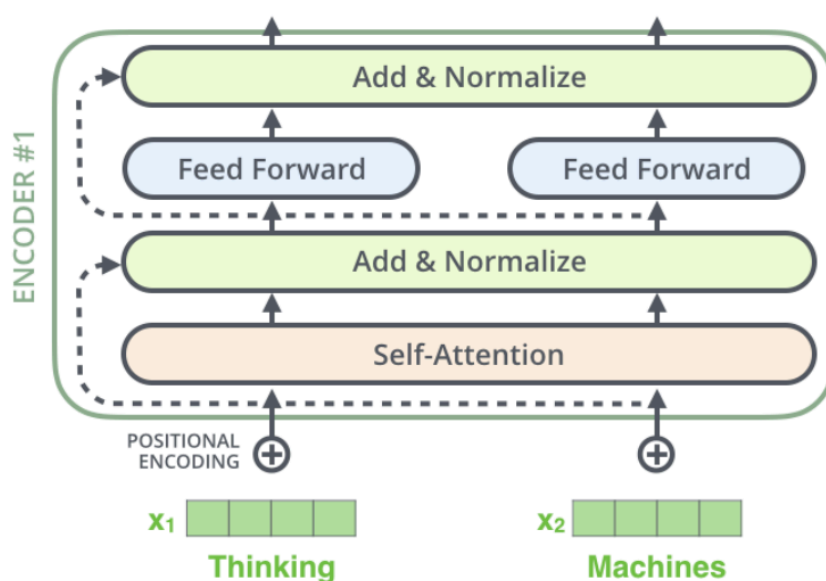
Copy

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

- pos = position (0, 1, 2, ...)
- i = dimension index (0 to d_model/2)
- d_model = embedding size (4 in the toy example)
- These values are **pre-computed** once and stay the same forever — they don't change during training.

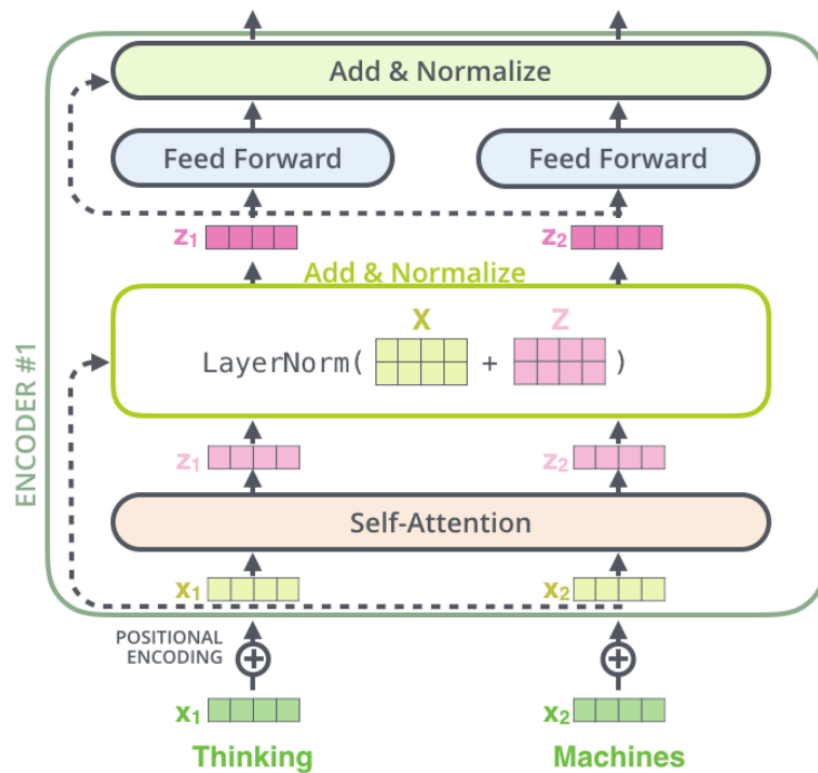
The Residuals

Each Sub-layer (self-attention , feedforward) in each encoder has a residual connection around it and is followed by layered-normalization step.

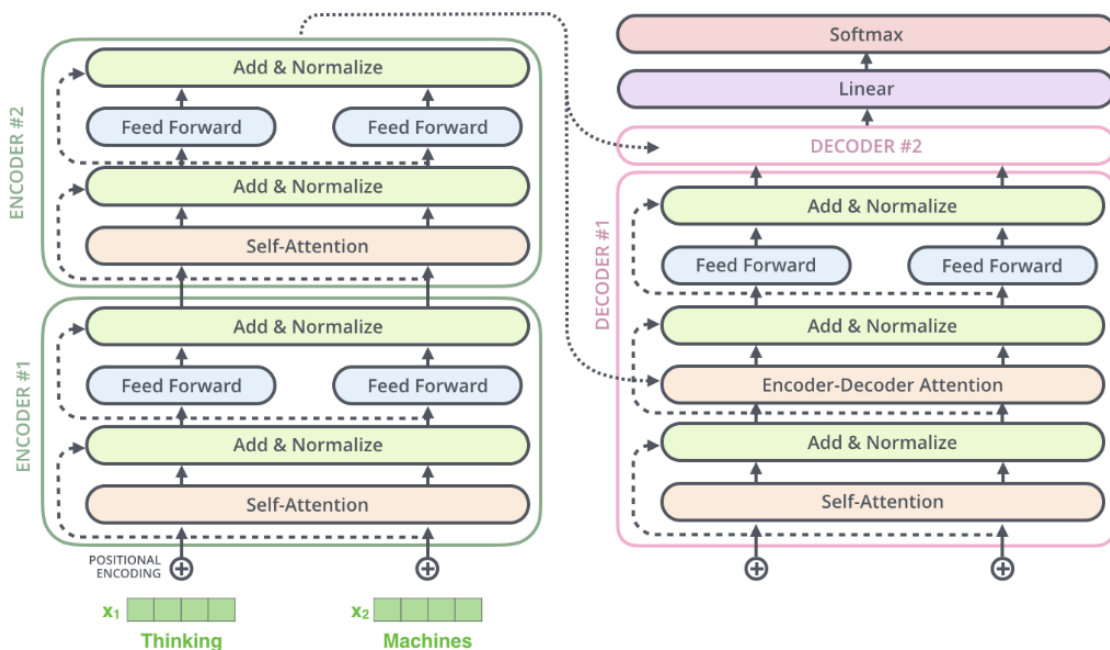


If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this:

If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this:

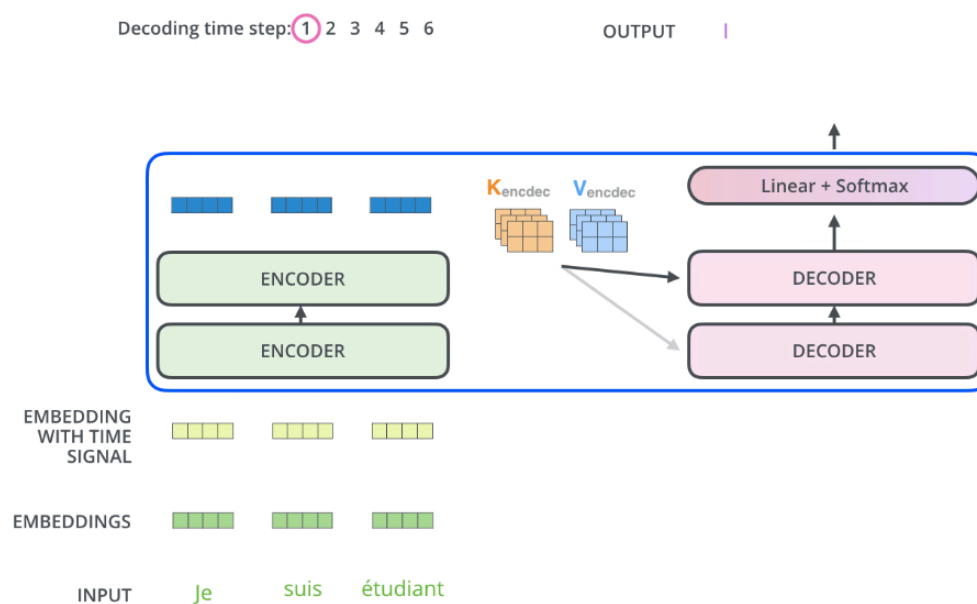


This goes for the sub-layers of the decoder as well. If we're to think of a Transformer of 2 stacked encoders and decoders, it would look something like this:

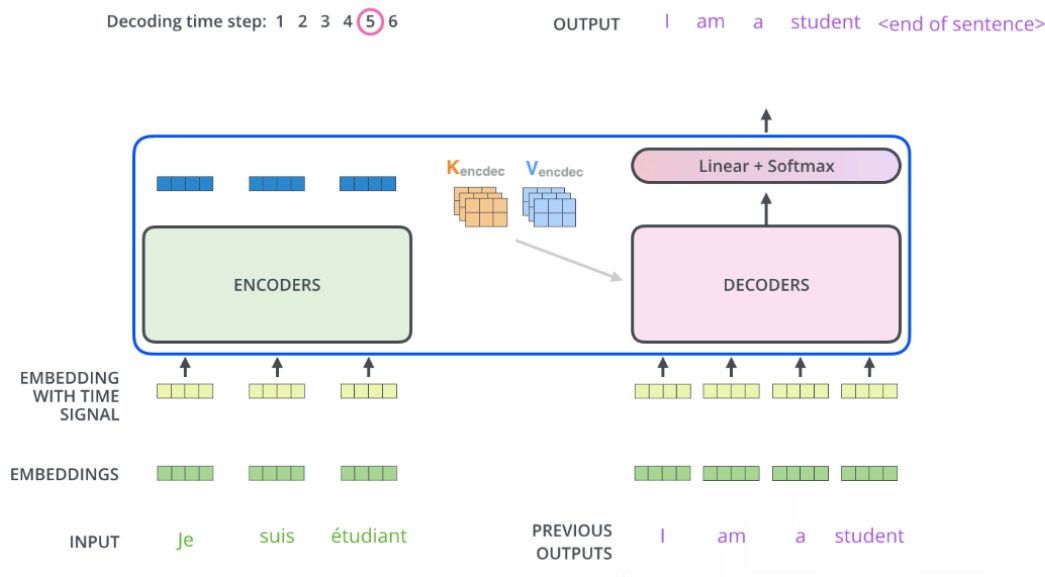


Decoder Side

Encoder starts by processing the input sequence, the output of the top encoder is then transformed into set of attention vectors K and V . These are to be used by each decoder in it's "encoder-decoder attention" layer which helps the decoder focus on appropriate places in the input sequence.



The steps repeats until a special symbol is reached indicating transformer decoder has completed it's output .The output of each step is fed to the bottom decoder in the next time step , and decoder pass their decoding up just like encoders did.



Self attention layer in decoder is slightly different than that of encoder. In decoder the self attention layer is only allowed to attend the earlier position in the output sequence .

This is done by masking the future positions before the soft max step.

Final Linear and Soft-max Layer

The decoder stacks outputs a vector of floats which is turned into words by the final linear and soft-max layer.

Linear Layer is a simple fully connected neural network that projects the vector produced by a stack of decoders into much much larger vector called "logits Vector".

Let's assume that our model knows 10,000 unique English words (our model's "output vocabulary") that it's learned from its training dataset. This would make the logits vector 10,000 cells wide – each cell corresponding to the score of a unique word. That is how we interpret the output of the model followed by the Linear layer.

The softmax layer then turns those scores into probabilities (all positive, all add up to 1.0). The cell with the highest probability is chosen, and the word associated with it is produced as the output for this time step.

Which word in our vocabulary
is associated with this index?

am

Get the index of the cell
with the highest value
(argmax)

5

log_probs



Softmax

logits



Linear

Decoder stack output



This figure starts from the bottom with the vector produced as the output of the decoder stack. It is then turned into an output word.